C73-A4

changze wu,shuang wu

September 2024

1 Q1

1.1 1

the line 6 should be x:=PowerOfTenToBinary($\frac{n}{2}$)

proof: for any i, $1 \leq i \leq n$, such that i,n are a power of 2, function PowerOfTenToBinary(i) will computes the binary representation of the number ten to nth power. by induction,

base case when i is 1 and it returns 1010 as wanted

suppose for any i such that $1 \leq i < n$, and i is a power of 2, it holds

then for the case when i = n: suppose n is the k-th power of 2. and let's suppose that t= PowerOfTenToBinary($\frac{n}{2}$)=PowerOfTenToBinary(2^{k-1}), then by using fastmult(t, t), which take two parameters are all binaries. Thus, fastmult(t, t), which t is binary representation by IH. Thus, the result of fastmult(t, t) = $t^{(10)_2}$ = PowerOfTenToBinary(2^k) which is the binary of the i-th power of 10, which is the nth power of 10,as wanted.

and for complexity, by master theory:

$T(n) = \begin{cases} 1 \cdot T(\frac{n}{2}) + c * n^{1.58}, \ when\ n > 1 \\ c, \ when\ n = 1 \end{cases}$ so the complexity is $n^{1.58}$// here we take $log_2 3$ as 1.58 which is the complexity of karatsuba algo

1.2 2

it will return sum(fastmult(Demicaltobinary(x_L),poweroftentobinary(len(x_L)),demicaltobinary(x_R)).

Proof : For any decimal number x, assume i = len(x), $1 \leq i \leq n$, i is power of 2.

base case: when i = 1, the algo directly return the binary[x], as wanted.

induction step: Suppose, when $1 \leq i < n$, the statement hold.WTP : when i = n, the state also hold. It is obvious that x = $x_L \times 10^{\frac{n}{2}} + x_R$, for len(x) = n and $x_L = \frac{n}{2}$. Since any integer in decimal numbers, we can do such things. According to this formula, we change it into algo language and each parameter in to binary form, we can get the same consequence with each in decimal form. QED.

complexity: by master theory , we can know to the a = 2, and b = 2. let us analysis the complexity on the last line it is $n^{1.58}$.

- first call the poweroftentobinary func, the len(x_L) = $\frac{n}{2}$, the running time is $\frac{n}{2}^{1.58} = n^{1.58}$

- second call the fastmult func with parameter, dec(x_L) and the binary form of $10^{\frac{n}{2}}$. We can obvious also get the running time is O($n^{1.58}$

- third call the sum func, we should consider the max length of fastmult func and dem func of x_R. Then we use loop to sum each position's digits, which call O(n)

Thus, O(n) + O($n^{1.58}$) + O($n^{1.58}$) = O($n^{1.58}$). Then we can apply this to master theory :

$$T(n) = \begin{cases} 2 \cdot T(\frac{n}{2}) + c * n^{1.58}, \ when\ n > 1 \\ c, \ when\ n = 1 \end{cases}$$ so the complexity is $n^{1.58}$

2 q2

let's create a class, maxPara, it has the following parameters:

lbig : max value left para of array

rbig : max value right para of array

ibig : max value of the current array

totalSum : total value of array

updating(A,l,r):

new maxPara res//create the result and we will return it

Algorithm 1 UPDATING($\mathcal{A}, l, r$)

```
1: if l=r then
2:    res.lbig,rbig,ibig,totalSum=A[l]
3:    return res
4: else
5:    mid = [(l+r)/2]
6:    leftMaxPara = updating(A,l,mid)
7:    rightMaxPara = updating(A,mid+1,l)
8:    res.lbig= max(leftMaxPara.totalSum+rightMaxPara.lbig,leftMaxPara.lbig)
9:    res.rbig = max(rightMaxPara.totalSum+leftMaxPara.rbig,rightMaxPara.rbig)
10:   res.ibig = max(leftMaxPara.rbig+rightMaxPara.lbig,leftMaxPara.ibig,rightMaxPara.ibig)
11:   res.totalSum = rightMaxPara.totalSum+leftMaxPara.totalSum
12:   return res
13: end if
```

mian function:

- res=updating(A,1,n)
- return res.ibig

correctness proof by induction : For any $1 \leq l \leq r \leq n$, the alog can always finding max value of sub-array inside of [l,r]

base case : it is obvious that when l=r there is only one element in this range so we hold.

induction step: Assume that there are r-l+1 = n-1 we hold the state. WTP : when A[1...n] we also hold the state. We consider three cases :

- CASE 1 the max subarray is in left. The wanted result is in the left side, so we will recursive the leftMaxPara. len(leftMaxPara) $\leq n$, by IH we will get the consequence as wanted.

- Case 2 the max subarray is in right. which is same with case 1.

- CAse 3 the max subarray need right part of left side and left part of right side. When the algo executes the len(A) = n, it should first run the leftMaxPara and rightMaxPara, which length are all less than n, by IH, all things are holds. Then the algo will return the res wich contain the lbig and rbig which are used in the original func, so res.ibig will execute the answer and return by the main func.As wanted.

All cases hold the state, QED. **complexity:** by master theremon $T(n) = \begin{cases} 2 \cdot T(\frac{n}{2}) + c * n^0, \ when\ n > 1 \\ c, \ when\ n = 1 \end{cases}$ so it is $n^{log_2 2} = n$ it is O(n)